

Process list & signals

When a build or simulation hangs, you need to see what is running and how to stop it

PID, exit status, and signals

- A process ID uniquely names a running program
- When a command finishes, it leaves an exit status, zero usually means success
- Signals are how you interrupt or stop a process
- Background jobs need kill with a PID, Control-C alone will not stop them

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / ps / kill

INTERACTIVE TOOL

Process list & signals

Read a ps table, stop a runaway with kill / Ctrl+C (SIGINT), and learn when you need kill -9 (SIGKILL).

Starter example: Foreground job is iverilog (PID 2201). Try ps, then Ctrl+C (SIGINT) to stop it. For hung_sim (2210), SIGINT is ignored — use kill -9 2210.

Load starter example

Challenges 0 / 22 cleared

Quiz: ps: ps stands for? Answer: process status or processes

Answer

Show hintCheckNextIdle

Quiz: psRun psForeground PIDQuiz: Ctrl+CCtrl+C iverilogQuiz: default killkill sleephung ignores INT

kill -9 hungQuiz: -9Quiz: catchableQuiz: prefergtkwave PIDkill gtwaveQuiz: Ctrl+C scope

No fg Ctrl+CSIGINT numberSIGKILL numberkill -2Count lab procsQuiz: STAT REvent log

Lab terminal

Foreground: PID 2201 · iverilog -o build/out.vvp rtl/top.v

Ctrl+C sends SIGINT to this process only.

Process table

Refresh

PID	PPID	USER	STAT	COMMAND
1	0	root	S	init
1000	1	lab	S	bash

Process list and signals lab

Real shell practice

```
wsl - module09-ps-kill-lab/examples/processes

# Track A - real Unix
# real shell session (Track A)

$ chmod u+x show_pid.sh fail.sh

$ ./show_pid.sh
PID of this process: 99746
Run 'echo $?' after this to see exit status (0 = success)

$ echo $?
0

$ ./fail.sh || echo $?
This script exits with status 1
1

$ ps aux | head -8 || true
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0  0.0 167332 12952 ?        Ss   Jul17   0:09 /sbin/init
root         2   0.0  0.0   2776   1928 ?        Sl   Jul17   0:00 /init
root         7   0.0  0.0   2776   132 ?        Sl   Jul17   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8
--pipe-fd 10 --log-truncate
root        60   0.0  0.0  56200 15580 ?        S<s  Jul17   0:07 /lib/systemd/systemd-journald
root        88   0.0  0.0  22380  6220 ?        Ss   Jul17   0:05 /lib/systemd/systemd-udevd
root        96   0.0  0.0 153004  2228 ?        Ssl  Jul17   0:00 snapfuse /var/lib/snapd/snaps/chromium_3375.snap
/snap/chromium/3375 -o ro,nodev,allow_other,suid
root        98   0.0  0.0 153004  2232 ?        Ssl  Jul17   0:00 snapfuse /var/lib/snapd/snaps/bare_5.snap
/snap/bare/5 -o ro,nodev,allow_other,suid

$ sleep 60 & kill $!; wait 2>/dev/null || true
```

Real shell — processes and signals

Real shell practice — try these

```
# chmod u+x ... — ensure the demo scripts are executable
chmod u+x show_pid.sh fail.sh
```

```
# ./show_pid.sh — run a script that prints its PID and exits
successfully
./show_pid.sh
```

```
# echo $? — print the last exit status (0 = success)
echo $?
```

```
# ./fail.sh — run a script that exits with status 1
./fail.sh
```

```
# echo $? — confirm a non-zero failure status
echo $?
```

Pitfalls to watch

- Do not kill processes you do not own or understand
- Prefer terminate, then escalate only if needed
- Remember that Control-C stops the foreground job, not an arbitrary background PID
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, finish a few challenges after the starter
- On the real shell, practice exit status, listing processes
- When you are ready, take the short quiz, then continue to job control